Enhancing Your User Experience

The Systematic Destruction of Professional Software Expertise Through Interface Redesign Cycles

Registry: [?]

Series Path: [HOWL-INFO-14-2026] → [HOWL-USER-1-2026] → [?]

DOI: 10.5281/zenodo.zzz

Date: November 2026

Domain: User Experience / Information Processing Theory

Status: Complete

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 4.6.

Abstract

Professional software interfaces change on release cycles that have no correlation with changes to the underlying functional capabilities. The phrase "enhancing your user experience" signals the replacement of a user's accumulated operational expertise with a new interface that serves the production metrics of a UI design team. This paper documents the pattern across seven major professional tools spanning 27 years, quantifies the retraining cost imposed on users, identifies the organizational incentive structure that drives the pattern, and demonstrates that the functional kernels of these applications have been stable for 10-25 years while their interfaces have been reorganized 3-8 times per decade. The paper concludes that the only structural defense against this pattern is to make the interface a user-owned data layer rather than a designer-shipped artifact.

---

1. The Inversion

The phrase "enhancing your user experience" contains a precise inversion. It refers to the replacement of one experience with another. The experience being replaced is the one the user built: the spatial memory that locates a button in 200 milliseconds, the keyboard shortcut chain that executes a five-step operation in under two seconds, the procedural workflow that moves from intent to output without conscious navigation. This accumulated experience is the user's primary asset when operating the software. It is the difference between a novice and an expert. It is the reason an expert produces more output per hour than a novice using the same tool.

The enhancement destroys this asset. The user's spatial memory is invalidated when panels move. The shortcut chain breaks when keystrokes are reassigned. The procedural workflow fails when dialogs are inserted, merged, or split. The expert is converted into a novice for a relearning period that ranges from days to weeks depending on the scope of the change.

The new experience that replaces the old one is not requested by the user. It is produced by a UI design team operating on a release cycle. The design team's output is the change itself. The change is the product. The user's relearning cost is the price.

Two definitions of "user experience" exist simultaneously within the organizations that produce professional software:

User experience (actual): The accumulated operational knowledge a user holds about how to accomplish their tasks within the software. This knowledge is stored in muscle memory, spatial memory, and procedural memory. It is built through thousands of hours of repetition. It is the reason experienced users are faster than new users. It is destroyed by interface changes and rebuilt through forced repetition with the new interface.

User experience (artifact): The designed interface that a UI team produces. It exists as wireframes, design system documentation, component libraries, and interaction specifications. It is measured by design system compliance scores, visual consistency audits, accessibility checklists, and first-time-user comprehension tests. It is the thing that appears in release notes and marketing materials.

These two definitions are in direct conflict. Optimizing the artifact requires changing the interface. Changing the interface destroys the actual experience. The organization measures the artifact. The user bears the cost of the destroyed experience. The cost is invisible to the organization because it is distributed across millions of users and does not appear in any metric the organization tracks.

---

2. The Stable Kernel

Before examining what changes, it is necessary to establish what does not change. The functional kernel of each major professional tool has been stable for a decade or more. The operations that users perform have not changed. The underlying algorithms, data structures, and processing pipelines have not changed in ways that require interface reorganization.

Product	Core Operations	Stable Since	Years Stable
Adobe Photoshop	Layers, selections, brushes, masks, transforms, color adjustments, filters	2003 (CS)	23

Product	Core Operations	Stable Since	Years Stable
Autodesk AutoCAD	Lines, arcs, circles, polylines, dimensions, hatches, blocks, layers, plot	2006	20
Unreal Engine	Actors, components, blueprints, materials, sequencer, build	2014 (UE4)	12
Unity	GameObjects, components, inspector, hierarchy, build	2018 (2018 LTS)	8
Blender	Modeling, sculpting, texturing, animation, rendering, compositing	2019 (2.80)	7
DaVinci Resolve	Media management, editing, color grading, mixing, delivery	2015 (Fusion integration)	11
Figma	Frames, components, auto layout, prototyping, export	2019	7

Genuine functional additions do occur. Photoshop added Generative Fill in 2023. Unreal Engine added Nanite and Lumen in 2022. Blender added Geometry Nodes in 2021. DaVinci Resolve added neural engine tools in 2023. These are real new capabilities that justify new interface elements because there is a new operation the user needs to access.

The additions are rare. Between Photoshop CS (2003) and Generative Fill (2023), twenty years passed with no functional change that required any interface reorganization. During those twenty years, the interface was reorganized approximately eight times. Each reorganization was described as an enhancement. None was required by a change in what the software does.

The functional kernel is the proof. If the operations the user performs have not changed, then any change to how the user accesses those operations is not a functional requirement. It is an organizational output.

3. The Task Audit

For each exemplar product, the following tables track the most common user operations across release versions. Each entry records the interaction path: the sequence of keystrokes, clicks, and visual scans required to complete the operation. Changes in interaction count that are not caused by changes in functionality are interface reorganization events.

3.1 Adobe Photoshop

Task	1999 (5.5)	2012 (CS6)	2015 (CC 2015)	2024 (CC 2024)
Export to PNG	`Alt+Shift+Ctrl+S` → PNG → Save (3 steps)	Same (3 steps)	File → Export → Export As → PNG → configure → Save (6 steps)	Same as 2015, or assign Quick Export shortcut manually (6 steps default)
Flatten and save	`Ctrl+Shift+S` → `Ctrl+S` (2 steps)	Same (2 steps)	Flatten → Save As → format dialog → options → confirm (5 steps)	Flatten → Save As vs Save a Copy disambiguation → format → options → confirm (5-6 steps)
Resize image	`Ctrl+Alt+I` → type width → OK (3 steps)	Same (3 steps)	Same (3 steps)	Same (3 steps)
New layer	`Ctrl+Shift+N` → Enter (2 steps)	Same (2 steps)	Same (2 steps)	Same (2 steps)
Adjust levels	`Ctrl+L` → adjust → OK (3 steps)	Same, Properties panel added (3 steps + visual noise)	Same (3 steps)	Same (3 steps)
Crop	`C` → drag → Enter (3 steps)	Same (3 steps, options bar reorganized)	Same (3 steps, options bar reorganized again)	Same (3 steps, options bar reorganized again)

Task	1999 (5.5)	2012 (CS6)	2015 (CC 2015)	2024 (CC 2024)
Refine selection edge	Select → Refine Edge → adjust → OK (4 steps)	Same (4 steps)	Select → Select and Mask → new workspace → adjust → OK (5 steps, new layout)	Same as 2015 (5 steps)
Apply Gaussian Blur	Filter → Blur → Gaussian Blur → adjust → OK (5 steps)	Same (5 steps)	Same (5 steps)	Same (5 steps)
Content-Aware Fill	Edit → Fill → Content-Aware → OK (4 steps)	Same (4 steps)	Same (4 steps)	Edit → Content-Aware Fill → new workspace → adjust → OK (5 steps)
Batch process	File → Automate → Batch → configure → OK (5 steps)	Same (5 steps)	Same (5 steps)	Same (5 steps)

The operations that gained steps (Export, Flatten+Save, Refine Edge, Content-Aware Fill) are operations where the UI team introduced a new dialog, workspace, or panel. The operations that remained unchanged (Resize, New Layer, Levels, Crop, Gaussian Blur, Batch) are operations where the UI team did not introduce a new interaction surface.

The functional capability behind each operation is identical across all versions. A PNG export in 1999 and a PNG export in 2024 produce the same output. The difference is the number of interactions required to reach that output.

3.2 Autodesk AutoCAD

Task	2000	2009 (Ribbon)	2020	2026
Draw line	`L` → click → click → Enter (4 steps)	Same (4 steps)	Same (4 steps)	Same (4 steps)
Draw circle	`C` → click → drag → click (4 steps)	Same (4 steps)	Same (4 steps)	Same (4 steps)

Task	2000	2009 (Ribbon)	2020	2026
Print drawing	`Ctrl+P` → configure → OK (3 steps, one dialog)	`Ctrl+P` → expand sections → configure → OK (4-5 steps, collapsible sections)	`Ctrl+P` → expand sections → configure → OK (reorganized again) (4-5 steps)	Same (4-5 steps)
Set current layer	Layer dropdown → select (2 steps)	Layer dropdown → select (2 steps, widget redesigned)	Same (2 steps, widget redesigned again)	Same (2 steps)
Insert block	`I` → select → OK → place (4 steps)	Same (4 steps)	Same (4 steps)	Same (4 steps)
Add dimension	`DIM` → click → click → place (4 steps)	Same (4 steps)	Same (4 steps)	Same (4 steps)
Export to PDF	Print → PDF printer → OK (3 steps)	Same (3 steps) or `EXPORTPDF` (typed, no shortcut)	Same (3 steps)	Same (3 steps)
Hatch area	`H` → click boundary → Enter (3 steps, dialog-based)	`H` → ribbon panel appears → click → Enter (3 steps, ribbon-based, layout changed)	Same (3 steps, ribbon reorganized)	Same (3 steps)
Edit polyline	`PE` → select → options → Enter (4 steps)	Same (4 steps)	Same (4 steps)	Same (4 steps)
Toggle ORTHO	F8 (1 step)	F8 (1 step, status bar icon changed from text to icon)	F8 (1 step, icon redesigned)	F8 (1 step)

The command-line operations (`L`, `C`, `I`, `DIM`, `PE`, `H`) are completely unchanged because the command line is a text interface. The UI team cannot redesign text. The GUI operations (Print, Layer dropdown, Hatch ribbon panel, status bar) were reorganized because they are visual surfaces the UI team can modify.

The command line is the existence proof that interface stability is possible. AutoCAD's command line has been stable for 44 years. The commands work the same way in 2026 as they did in 1982. The Ribbon, introduced in 2009 as a "modern" replacement for toolbars and menus, has been reorganized four times in 17 years.

3.3 Unreal Engine

Task	UE4 (2014)	UE5.0 (2022)	UE5.4 (2026)
Place actor	Content Browser → drag to viewport (2 steps)	Content Browser → drag to viewport (2 steps, browser redesigned)	Same (2 steps)
Build lighting	Toolbar → Build → Build Lighting Only (2 clicks)	Menu → Build → Build Lighting Only (button removed from toolbar) (3 clicks)	Menu → Build → submenu → Build Lighting Only (4 clicks)
Create Blueprint	Content Browser → right-click → Blueprint (3 clicks)	Same (3 clicks, browser redesigned)	Same (3 clicks)
Edit Blueprint variable	Details → variable row → edit (2 clicks)	Details → variable row → edit (2 clicks, panel reorganized)	Same (2 clicks)
Create material	Content Browser → right-click → Material (3 clicks)	Same (3 clicks)	Same (3 clicks)
Package project	File → Package → platform (3 clicks)	Platforms → platform → Package (3 clicks, menu restructured)	Same (3 clicks)
Create button widget	Palette → drag Button → set style (3 steps + minimal config)	Palette → drag Button → configure Normal/Hovered/Pressed/Disabled/Focused styles (3 steps + 5-9 state configs)	Same (3 steps + 9 state configs)
Play in editor	Toolbar → Play (1 click)	Toolbar → Play (1 click, button moved slightly)	Same (1 click)

The button widget row is examined in detail in Section 5.

3.4 DaVinci Resolve

Task	Resolve 14 (2017)	Resolve 17 (2021)	Resolve 19 (2024)
Color grade clip	Color page → select → wheels (2 steps)	Same (2 steps)	Same (2 steps)
Cut clip	`Ctrl+B` (1 step)	Same (1 step)	Same (1 step)
Add transition	Drag from panel (2 steps)	Same (2 steps)	Same (2 steps)
Export timeline	Deliver → format → Add → Start (4 steps)	Deliver → format → Add → Start (4 steps, page reorganized)	Deliver → format → Add → Start (4 steps, page reorganized again) or Quick Export (no default shortcut)
Import media	Media page → drag files (2 steps)	Same (2 steps, page reorganized)	Same (2 steps, page reorganized again)
Add audio effect	Fairlight → select track → effects rack (3 steps)	Same (3 steps, rack moved)	Same (3 steps, rack moved again)

The color grading operations have zero interaction changes across 7 years. The color page is Resolve's competitive advantage. Colorists are the users who choose Resolve over competitors. The UI team does not touch the color page because doing so would cause measurable user attrition. Every other page has been reorganized. This selectivity proves the UI team makes strategic decisions about which users can absorb retraining cost and which users would leave.

---

4. The Export Problem

The final operation in every creative workflow is getting output out of the application. The user has finished their work. They need to produce a deliverable file. This operation sits between "work is done" and "work is delivered." Every additional interaction on this operation is pure friction between completion and delivery.

Across every major creative application, the export operation has become slower over time.

4.1 Photoshop: Export to PNG

1999: `Alt+Shift+Ctrl+S` opened Save for Web. The dialog showed a preview and format options. The user selected PNG, clicked Save, chose a location, and confirmed. The shortcut was four keys held simultaneously, then two clicks. The dialog remembered the last-used format. A user who always exported PNG opened the dialog with the format already set. Total: 1 shortcut + 1 click + 1 file dialog.

2015: Save for Web was deprecated. The label "(Legacy)" was appended to the menu entry. The promoted replacement was Export As, accessed through File → Export → Export As. Export As opens a larger dialog with format selection, canvas size controls, metadata toggle, color space options, and a preview that re-renders on every settings change. The dialog does not remember the last-used format across sessions in all configurations. No default keyboard shortcut was assigned to Export As.

The user who exports to PNG 50 times a day went from a four-key shortcut to a three-level menu navigation. The menu path is: move mouse to File (menu bar), click, move to Export (submenu), hover to expand, move to Export As, click. This is 5 pointer movements and 2 clicks before the dialog opens. The dialog then requires format selection (1 click), and Save (1 click + file dialog). Total: 7 interactions minimum, versus 3 with the shortcut.

2017: Quick Export as PNG was added under File → Export → Quick Export as PNG. This performs a direct PNG export with no dialog, using the last-used settings. No default keyboard shortcut was assigned. The user can assign one through Edit → Keyboard Shortcuts → File → Export → Quick Export as PNG. The assignment path is: Edit (menu bar) → Keyboard Shortcuts → scroll or search to find "Quick Export as PNG" → click the shortcut field → press the desired key combination → click Accept → click OK. This is approximately 8 interactions to configure a shortcut that should have been default.

2024: The state is unchanged. Quick Export exists. It has no default shortcut. A fresh Photoshop installation on a new machine has no fast path to PNG export. The user must either configure Quick Export's shortcut (8 interactions, repeated per machine) or use Export As (7+ interactions per export).

The progression: a three-interaction operation in 1999 became a seven-interaction operation in 2024, with an optional configuration step of eight interactions that must be repeated on every new installation to restore the original speed.

4.2 Illustrator: Export to SVG

2005 (CS2): File → Save As → SVG. The SVG Options dialog had four settings: SVG Profile, Fonts, Images, and CSS Properties. The dialog was one panel. The user set options once and the dialog remembered them.

2024: File → Export → Export As → SVG. The SVG Options dialog now has: Styling (Presentation Attributes / Internal CSS / Inline Style), Font (SVG / Outline / Converted), Images (Embed / Link), Object IDs (Layer Names / Minimal / Unique), Decimal (1-7), Minify, Responsive, and additional output options. Twelve configurable fields. The dialog does not consistently remember settings across sessions. The user who wants the same SVG output every time must verify twelve fields every time.

4.3 Premiere Pro: Export Video

2017 (CC 2017): File → Export → Media opened the Export Media dialog. One panel with Source, Output, and codec settings. Format dropdown, preset dropdown, output path. Configure and click Export.

2023 (CC 2023): The Export Media dialog was replaced with a full-page Export workspace. The workspace has a source monitor, output preview, format/preset selectors, encoding settings, and a render button distributed across multiple panels. The workspace layout is different from the previous dialog. Users who knew the dialog layout had to relearn positions. The workspace offers more simultaneous visibility of settings but requires more screen space and more visual scanning to locate specific options.

The operation is: "export this timeline as H.264 MP4." The functional requirement has not changed. The interface surface area approximately doubled.

4.4 Blender: Render and Save

2.79 (2018): `F12` rendered the current frame. In the render window, `F3` saved the result. Two keystrokes for the complete render-and-save workflow.

2.80 (2019): `F12` still renders. `F3` was reassigned to the operator search menu. Saving the render result requires: in the render window, Image menu → Save As → file dialog. The two-keystroke workflow became a keystroke plus three interactions.

4.x (2026): Same state. `F3` remains assigned to operator search. No replacement shortcut was assigned for saving the render result. The user can assign one through Edit → Preferences → Keymap → Image Editor → Image → Save As Image → assign shortcut. This configuration path is approximately 8 interactions deep.

4.5 The Pattern

Product	Operation	Original Steps	Current Steps	Steps Added
Photoshop	Export PNG	3	7 (or 3 after 8-step config)	+4 (or +8 once)
Illustrator	Export SVG	5 (4 memorable options)	7 (12 options to verify)	+2 steps, +8 cognitive
Premiere Pro	Export MP4	5 (one dialog)	7 (full workspace)	+2
Blender	Render + Save	2	5 (or 2 after 8-step config)	+3 (or +8 once)
Resolve	Export timeline	4	4 (page reorganized) or Quick Export (no shortcut)	+0 steps, +2 relearning events
AutoCAD	Print to PDF	3	5 (expanded sections)	+2
Figma	Export frame	2 (panel visible)	4 (panel collapsed, must expand)	+2

No product made its export operation faster across any major release. Every product either added steps, removed a shortcut, or reorganized the interface for the export operation. The operation that the user performs most frequently at the end of every work session — the operation where speed has the highest impact on workflow — is the operation that has consistently become slower.

---

5. The Button Problem

Unreal Engine's UMG widget system provides the most concentrated example of design-system complexity imposed on users. A button is the most fundamental interactive element in any user interface. The user's requirement is: a rectangular region that responds to a click by executing a function, with visual feedback that it can be clicked.

5.1 What the User Wants

A button requires two pieces of information to function: what it looks like and what it does when clicked. In the simplest case, the user provides a background color, a text label, and a click handler. The button should display the label on the background, change appearance slightly when the mouse hovers over it, change appearance when the mouse presses it, and execute the handler when the mouse releases on it.

5.2 What UE Requires

UE's Slate/UMG button widget exposes the following states, each requiring independent visual configuration:

State	Description
Normal	Default appearance, no interaction
Hovered	Mouse cursor is over the button
Pressed	Mouse button is held down on the button
Disabled	Button is non-interactive
Focused	Button has keyboard focus
Normal + Selected	Button is in selected state, no hover
Hovered + Selected	Button is in selected state, mouse hovering
Pressed + Selected	Button is in selected state, mouse pressing
Disabled + Selected	Button is in selected state, non-interactive

Each state requires a style definition containing: background brush (which may be a solid color, a texture, a material instance, or a nine-slice image), tint color (RGBA), outline color, outline width, corner radius, padding (four edges independently), margin (four edges independently), minimum desired size, maximum desired size, foreground color, and optionally a sound event.

5.3 The Configuration Burden

For a button with textured backgrounds (the standard in shipped game UI), each of the nine states requires an image asset. A button bar with five buttons requires 45 image assets. A menu screen with twenty buttons across multiple groups requires 180 image assets. A full game UI with eight menu screens, each containing 15-25 buttons, requires 1,080 to 1,800 image assets for buttons alone, before considering other widget types (checkboxes, sliders, tabs, text inputs).

For a button with solid-color backgrounds (the standard in tools and prototypes), each state requires manual color specification. There is no auto-derivation system that generates hover (lighter), pressed (darker), and disabled (desaturated) states from a base color. The user must manually specify each color for each state. Five buttons with solid-color backgrounds require 45 manual color entries.

5.4 The Design System Justification

The justification for this configuration surface is accessibility and polish. A shipped game UI should respond to all interaction states with distinct visual feedback. Keyboard focus indication is an accessibility requirement. Disabled state indication prevents user confusion. Selected state indication provides context in toggle button groups.

This justification is valid for the final polish pass of a shipped AAA game's player-facing UI. It is not valid for:

- Prototyping (where buttons need to work, not look finished)
- Editor tools (where developers build internal tools with minimal UI investment)
- Game jams (where the entire game is built in 48 hours)
- Early development (where gameplay mechanics are tested through placeholder UI)
- Any project where the team does not include a dedicated UI artist

The system does not offer a graduated path. There is no "simple button" that requires two configuration values (background color, text color) and auto-derives the remaining states. The user must configure the full state matrix or accept the default style, which is a flat gray rectangle that does not match any project's visual identity.

5.5 The Specialization Requirement

The complexity of the button configuration system means that configuring a game's UI requires specialized knowledge. A gameplay programmer who can implement combat systems, AI behavior trees, and networked replication cannot set up a menu screen without spending hours learning the UMG widget style system.

Studios respond by hiring dedicated UI artists and UI programmers. These are roles whose entire purpose is configuring widget styles, creating UI materials, and building UMG widget blueprints. The roles exist because the tool's complexity requires them. The tool's complexity is maintained because the roles exist to justify it. The UI team

within Epic that designs the UMG system builds for the complexity requirements of the UI artists at AAA studios who provide feedback. The feedback loop reinforces the complexity because the people providing feedback are the people whose jobs depend on the complexity.

5.6 The Simple Alternative

A system that solves the same problem with minimal configuration:

The user specifies two values: background color and text color. The system derives:

State	Derivation
Normal	Background color as specified
Hovered	Background color lightened 10%
Pressed	Background color darkened 10%
Disabled	Background color at 50% saturation, 70% opacity
Focused	Normal + 2px outline in text color
Selected	Background color shifted 15% toward a highlight hue
Hovered + Selected	Selected color lightened 10%
Pressed + Selected	Selected color darkened 10%
Disabled + Selected	Selected color at 50% saturation, 70% opacity

Any state can be individually overridden. The system provides the simple case as the default and the complex case as an opt-in override. This approach covers approximately 90% of use cases with two configuration values. The remaining 10% of cases override individual states as needed.

UE does not provide this. Every state must be explicitly configured. The 90% case pays the cost of the 10% case on every button.

---

6. The Incentive Machine

The pattern documented in Sections 2-5 is not accidental. It is the output of an organizational incentive structure that requires interface changes on a release cycle.

6.1 The UI Team's Production Metric

A UI design team inside a software company must produce measurable output. The output must be visible. Performance improvements are credited to engineering. Bug fixes are credited to QA. Backend architecture improvements are credited to platform engineering. The only artifacts that are visibly and exclusively the UI team's work are changes to the interface the user sees.

If the UI team ships a release with no visible changes, the team produced no measurable output. A team with no measurable output is reduced in the next budget cycle. The UI

team's continued existence requires visible interface changes on every release cycle.

This is not a failure of individual designers. It is a structural incentive. The designers may be talented, thoughtful, and genuinely concerned with user welfare. The organizational structure requires them to produce changes regardless of whether changes are needed. A designer who says "the current interface is optimal, I recommend no changes" is a designer who produced no output for the release cycle.

6.2 The Measurement Gap

The metrics used to evaluate UI changes do not capture the cost of the changes.

Metrics that are measured:

Metric	What It Captures	Who Benefits
First-time user task completion rate	Whether new users can find operations	New users
Visual consistency score	Whether the interface matches the design system	Design team
Accessibility compliance	Whether the interface meets WCAG guidelines	Users with disabilities
User survey (post-release)	Immediate reaction to new interface	Marketing (positive quotes)
A/B test (new users)	Whether new layout improves new user performance	New users

Metrics that are not measured:

Metric	What It Would Capture	Who Bears the Cost
Power user velocity change	Whether experienced users are faster or slower	Power users
Shortcut retention rate	Whether existing shortcuts still work	All existing users
Relearning time	Hours spent rebuilding spatial and procedural memory	All existing users
Custom configuration loss	Whether user-configured workspaces, shortcuts, or presets survived the update	Power users
Documentation update cost	Hours spent updating internal training materials	Teams and organizations
Error rate during relearning	Whether users make more mistakes during the transition period	All existing users

The metrics that justify the change measure the new interface's performance with users who have no prior experience. The metrics that would quantify the cost measure the change's impact on users who had prior experience. The first set is measured. The second set is not. The decision to ship the change is made using only the first set.

6.3 The Subscription Coupling

Subscription licensing models remove the user's ability to remain on a version that works for them.

Adobe Creative Cloud requires the current version. An organization paying for Creative Cloud receives interface changes with every update. There is no option to pay for Creative Cloud and receive only bug fixes and performance improvements without interface reorganization. The functional features (Generative Fill, Neural Filters) are bundled with the interface changes. The user who wants the new features must accept the new interface.

Autodesk subscriptions follow the same model. AutoCAD subscribers receive the current version with the current ribbon layout. There is no "keep my current ribbon but give me the new DWG features" option.

Unreal Engine and Unity are free to use but couple interface changes with engine updates. A studio that needs a bug fix in the rendering pipeline must update to a version that includes interface reorganization. There is no patch-only update path that fixes the renderer without moving the Build button.

This coupling ensures that the retraining cost cannot be avoided by users who need the software. The user's only options are: accept the retraining cost, or stop using the software. For professional users whose livelihood depends on the software, stopping is not an option. The retraining cost is mandatory.

6.4 The Fashion Cycle

Professional software UI converges toward the same visual and interaction patterns because UI teams reference each other's work.

When Apple introduced flat design in iOS 7 (2013), every professional software application followed within 2-3 years. Skeuomorphic icons were replaced with flat icons. Shadow effects were removed. Color palettes shifted to muted tones. Border radii increased. These changes were described as "modernization." The modernization was not driven by user research showing that flat icons improved task performance. It was driven by the professional consensus among UI designers that flat design was the current best practice. The consensus was self-referential: flat design was the best practice because leading platforms adopted it, and leading platforms adopted it because it was the best practice.

When Google introduced Material Design (2014), another convergence cycle began. Cards, floating action buttons, and bottom navigation bars appeared in applications that had no relationship to mobile interfaces. AutoCAD's desktop ribbon acquired visual elements borrowed from Material Design. Photoshop's panel headers adopted rounded

corners and increased padding. The changes served no functional purpose for desktop applications used with mouse and keyboard.

The fashion cycle ensures that even if a UI team wanted to maintain interface stability, they would face organizational pressure to adopt the current design fashion. A product whose interface looks "dated" (meaning it looks like last year's design consensus) is perceived as unmaintained regardless of its functional capabilities. The perception drives upgrade decisions. The upgrade decisions drive revenue. The revenue justifies the UI team's next cycle of changes.

---

7. The Relearning Tax

The cost of interface reorganization falls entirely on the user. It is never measured by the organization that imposes it.

7.1 Cost Categories

Discovery cost: The user must find where an operation moved. For an operation previously located by spatial memory in under 200 milliseconds, the discovery scan takes 2-30 seconds. The user must visually scan menus, toolbars, and panels looking for a label or icon they recognize in a new position. This cost is paid once per moved operation per release cycle.

Spatial memory rebuild: After discovering the new location, the user must build new spatial memory. Research on motor learning indicates that procedural memory for spatial targets requires 10-50 repetitions to become automatic. During the rebuild period, each use of the operation includes a conscious recall step that adds 1-3 seconds. For an operation used 20 times per day, the rebuild period costs 20-60 additional seconds per day for 5-10 working days, totaling 100-600 seconds (approximately 2-10 minutes) per operation.

Shortcut reassignment: If a keyboard shortcut was removed, reassigned, or not assigned to a new feature, the user must either accept a slower workflow or configure a custom shortcut. The configuration path in most applications is 5-10 interactions deep in a preferences dialog. The custom shortcut must be reconfigured on every new machine, fresh installation, or workspace reset. The configuration investment is not portable.

Procedural workflow reconstruction: If the change altered a multi-step workflow (inserted a dialog, split a command, merged two commands, changed the order of options in a dialog), the user must reconstruct the procedural memory for the entire workflow. A five-step workflow where step 3 changed requires relearning the full five-step sequence because procedural memory stores sequences as chains, not as independent steps. A broken chain must be rebuilt from the break point forward.

Documentation and communication: Organizations with internal training materials, recorded tutorials, or documented procedures must update all references to the changed operations. A studio with 50 artists using shared Photoshop workflows must communicate

every change and update every document. This cost is paid once per organization per release cycle but scales with organization size.

7.2 Per-User Cost Per Redesign Cycle

Cost Category	Time Per Occurrence	Occurrences Per Major Redesign	Subtotal
Discovery	5-30 seconds	10-50 moved operations	1-25 minutes
Spatial memory rebuild	2-10 minutes per operation	10-50 moved operations	20-500 minutes
Shortcut reconfiguration	1-5 minutes per shortcut	5-20 reassigned shortcuts	5-100 minutes
Procedural workflow reconstruction	5-30 minutes per workflow	3-10 affected workflows	15-300 minutes
Documentation update	30-120 minutes per team member	1	30-120 minutes

Conservative total per user per major redesign: 1-8 hours.

Moderate estimate per user per major redesign: 3-12 hours.

7.3 Industry Scale

Product	Estimated Users	Redesign Cycles Per Decade	Per-Cycle Cost (hours, moderate)	Decade Total (person-hours)	Decade Total at $40/hr
Photoshop	30 million	3-4	6	540-720 million	$21.6-28.8 billion
AutoCAD	5 million	4	8	160 million	$6.4 billion
Unreal Engine	3 million	2	4	24 million	$960 million
Unity	4 million	3	5	60 million	$2.4 billion
Blender	5 million	2	3	30 million	$1.2 billion
DaVinci Resolve	3 million	3	4	36 million	$1.44 billion
Figma	10 million	2	3	60 million	$2.4 billion

Estimated total industry-wide retraining cost per decade: $36-43 billion.

This cost does not appear in any financial statement. It does not appear in any product

metric. It is distributed across millions of users and millions of organizations. No single user's 6-hour retraining cost is large enough to cause them to switch products. The cost is small enough per user to be absorbed, but large enough in aggregate to represent a significant transfer of value from users to software organizations.

7.4 The Regressive Distribution

The retraining cost is regressive. It falls hardest on the users who were most proficient with the previous interface.

A new user who learns the application after the redesign pays zero retraining cost. They build spatial memory and procedural workflows for the current interface. Their experience of the redesign is neutral to positive (the interface is "modern" and matches current design conventions).

A casual user who uses the application infrequently has weak spatial memory. Their retraining cost is low because they were already slow. They navigate by reading labels and scanning menus. The redesign moves the labels, but their strategy (read and scan) still works at approximately the same speed.

A power user who uses the application eight hours per day has strong spatial memory, fast shortcut chains, and procedural workflows that execute without conscious thought. The redesign invalidates all three. The power user is forcibly converted to casual-user speed for a relearning period of days to weeks. During this period, their output drops to the level of someone who started using the application recently.

The power user is the person who produces the most output per unit of time with the application. The retraining cost falls disproportionately on the person whose time generates the most value. The UI change that was justified by metrics measured on new and casual users imposes its highest cost on the most productive users.

---

8. The Vocabulary of Destruction

The language used to describe interface changes consistently frames destruction as creation. Each term has a precise operational meaning that differs from its marketed meaning.

8.1 Glossary

"Streamlined"

Marketed meaning: Made simpler and more efficient.

Operational meaning: Controls were removed from the default view. Features that were previously accessible in one step now require expanding a hidden section, navigating a submenu, or opening a preferences panel. The interface has fewer visible elements, which makes screenshots look cleaner and first-time-user comprehension tests score

higher. Power users who used the removed controls must now perform additional steps to access them.

Example: Photoshop CC 2015 "streamlined" the Export workflow by replacing Save for Web (one dialog, all options visible) with Export As (larger dialog, some options hidden behind disclosure triangles).

"Modernized"

Marketed meaning: Updated to reflect current standards.

Operational meaning: The visual language changed to match the current design fashion. Icons were redesigned. Colors were updated. Spacing and padding were adjusted. Typography was changed. None of these changes affect functionality. All of them invalidate visual scanning patterns that users built to locate operations by icon shape, color, and position.

Example: AutoCAD 2015 "modernized" the status bar by replacing text labels (SNAP, GRID, ORTHO) with icons. Users who found ORTHO by reading the word "ORTHO" at the bottom of the screen now had to learn which icon represented ORTHO.

"Simplified"

Marketed meaning: Made easier to use.

Operational meaning: Options were hidden behind an expander, disclosure triangle, or "Advanced" tab. The options were not removed because removing them would cause complaints. They were hidden so the default view contains fewer elements. Users who need the hidden options must click an additional control on every use. The simplification adds a step for experienced users to achieve the same result.

Example: AutoCAD's Plot dialog added collapsible sections in 2009. Settings that were previously visible in the dialog are now hidden until the user expands each section. The user who needs to change the plot scale must first expand the "Plot Scale" section, which was previously always visible.

"Unified"

Marketed meaning: Combined separate workflows into one coherent experience.

Operational meaning: Two operations that had distinct, optimized interfaces were merged into one interface that handles both cases. The merged interface asks questions relevant to case A when the user is performing case B, and vice versa. The user must dismiss irrelevant options on every use.

Example: Photoshop CC 2021 split "Save As" into "Save As" and "Save a Copy." The previous single operation now requires the user to choose between two commands, each of which opens a different dialog with different default behaviors.

"Redesigned"

Marketed meaning: Improved through design.

Operational meaning: The spatial layout changed. Panels moved to different positions. Buttons relocated within panels. Menu structures reorganized. The word "redesigned" implies the new design is better than the old design. The operational effect is lateral: the same operations exist in different positions. The user's spatial memory for the previous positions is invalidated.

Example: DaVinci Resolve 18 "redesigned" the Deliver page. The format selector, codec selector, and filename field changed positions. Users who had spatial memory for "format is in the top left" had to re-scan the page to find the format selector in its new position.

"Improved workflow"

Marketed meaning: The sequence of operations is now more efficient.

Operational meaning: Additional steps were inserted between the user's intent and the operation's completion. A preview panel, a confirmation dialog, a format selection step, or a metadata toggle was added to a previously direct path. Each additional step is individually defensible: the preview helps the user verify output before committing, the confirmation prevents accidental operations, the format selection ensures correct output. Collectively, the additional steps transform a fast operation into a slow one. The user who never needs the preview must dismiss it on every use.

Example: Photoshop's Content-Aware Fill was Edit → Fill → Content-Aware → OK (4 steps). CC 2019 "improved the workflow" by introducing a Content-Aware Fill workspace with a preview panel, sampling area controls, and output settings. The operation is now 5+ steps with a mandatory preview rendering delay.

"Updated to reflect current best practices"

Marketed meaning: Brought into alignment with industry standards.

Operational meaning: The UI team examined what other UI teams shipped and adopted the same patterns. The best practices are not derived from user research on productivity. They are derived from the professional consensus of UI designers about what interfaces should look like. This consensus is self-referential: best practices are defined by what leading products do, and leading products follow best practices.

Example: Multiple applications adopted bottom navigation bars, card-based layouts, and hamburger menus after Google's Material Design popularized them. These patterns were designed for mobile touch interfaces. They were applied to desktop applications used with mouse and keyboard, where they are less efficient than the toolbar and menu patterns they replaced.

---

9. The Stable Alternative

Interface stability is not a theoretical ideal. Stable interfaces exist and have been maintained for decades. They share one structural property: the interface is either a text protocol that cannot be visually redesigned, or a user-owned data layer that no release cycle modifies.

9.1 Text Protocol Interfaces

AutoCAD command line (1982-2026, 44 years): The command `LINE` draws a line. The command `CIRCLE` draws a circle. The command `PLOT` prints. These commands have been unchanged for 44 years. A user who learned them in 1982 uses them identically in 2026. The command line is immune to UI redesign because it is a text protocol. There is no visual surface for a UI team to reorganize. The characters L-I-N-E cannot be rearranged into a "more modern" sequence.

Power users who type commands in AutoCAD are immune to every ribbon reorganization, toolbar restructure, and status bar redesign that has occurred over 26 years. Their workflow speed is identical across every version. The command line is the stable kernel that proves interface stability is possible.

Terminal/shell interfaces (1970s-2026): `ls` lists files. `cd` changes directory. `grep` searches text. `gcc` compiles code. These commands have been stable for 50 years. Users who learned them decades ago use them identically today. No UI team has reorganized the shell because there is no visual surface to reorganize.

Vim keybindings (1991-2026, 35 years): `dd` deletes a line. `yy` copies a line. `p` pastes. `:wq` saves and exits. A user who learned these bindings in 1991 uses them identically in 2026. The bindings have never been reorganized because they are a user-configured text protocol.

Emacs keybindings (1976-2026, 50 years): `C-x C-s` saves. `C-x C-f` opens a file. `C-w` cuts. `M-w` copies. `C-y` pastes. These bindings have been unchanged for 50 years. The user's configuration file (`.emacs` or `init.el`) persists across versions. No release has ever modified a user's configuration file.

9.2 User-Owned Data Interfaces

The alternative to a designer-shipped interface is a user-owned data interface. In this model, the interface is not an artifact of the release. It is an artifact of the user's data. The software provides a rendering pipeline that reads user data and produces an interface. The rendering pipeline has no opinion about where panels go or what buttons look like. The user's data holds those opinions.

A system built on this principle has the following properties:

Panel positions are data fields. Which panel appears where, what size it is, and whether it is visible are stored in a user-owned record. No release cycle can change these fields because they belong to the user, not to the software.

Keyboard shortcuts are data entries. Each shortcut maps a key combination to a command string. The mapping is stored in a user-owned record. No release cycle can reassign shortcuts because the assignments belong to the user.

Visual styling is a data record. Colors, fonts, spacing, borders, and corner radii are stored in a theme record that the user can read and edit. No release cycle can change the visual language because the visual language is user data.

Tool selection is a data field. Which tool populates which panel area is stored in a user-owned record. The user can put any tool's controls in any panel. No release cycle can reorganize which controls appear where because the organization is user data.

The rendering pipeline is the only shipped artifact. The pipeline reads UiPanel trees and converts them to layout elements. It applies theme data to determine colors, fonts, and spacing. It reads command strings from panel definitions and executes them when buttons are clicked. The pipeline itself is functional infrastructure that does not express opinions about interface organization.

In this model, the UI team cannot "enhance the user experience" because the user experience is not the UI team's artifact. The user experience is the user's data. Enhancing it requires the user to change their own data, which they do voluntarily when they want a different workspace, and never involuntarily because of a software update.

---

10. Conclusion

"Enhancing your user experience" is a value transfer operation. It transfers accumulated expertise from the user (who loses spatial memory, shortcut chains, and procedural workflows) to the UI team (who gains production metrics and release note entries) and to the business (who gains upgrade justification and subscription renewal rationale).

The transfer is invisible because the cost is distributed across millions of users and the benefit is concentrated in a single organization's quarterly reports. Each individual user absorbs a relearning cost of 1-12 hours per major redesign cycle. Multiplied across the user bases of the seven exemplar products, the aggregate cost is approximately $36-43 billion per decade.

The functional kernels of these applications have been stable for 10-25 years. The operations users perform have not changed. The interfaces through which they access those operations have been reorganized 3-8 times per decade. None of the reorganizations were required by functional changes. All of the reorganizations imposed retraining costs on users.

The retraining cost is regressive. It falls hardest on power users, the people who generate the most output per hour and whose accumulated expertise represents the largest investment. The metrics that justify the changes are measured on new users who have no expertise to lose. The people who bear the cost are not represented in the decision.

The alternative exists. Text protocol interfaces have been stable for 35-50 years. User-owned data interfaces make the workspace a persistent record that no release cycle can modify. The structural defense against the enhancement cycle is to remove the interface from the set of artifacts that a UI team ships and place it in the set of data that a user owns.

The question is not whether stable interfaces are possible. AutoCAD's command line and Vim's keybindings prove they are. The question is whether the organizational incentive to ship visible changes on every release cycle can coexist with the user's need for interface

stability. The evidence from 27 years across seven major products suggests it cannot. Where the UI team has authority to change the interface, the interface changes. Where the UI team has no authority (text protocols, user-owned configuration), the interface is stable. The user's defense is structural: own the interface as data, and no one can enhance your experience without your consent.

[**HOWL-INFO-14-2026**] Bits and Ops. Github: https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-14-2026

[**HOWL-USER-1-2026**] Layout Is Not a Feature. Github: https://github.com/ghowland/papers/tree/main/papers/USER/HOWL-USER-1-2026